OS — Assignment 10 Report

Student Name : Konyala Sri Charan Reddy

Roll Number : 2401MC53

1. Summary

This report explains the implementation details, kernel modifications, and experimental results for Operating Systems Assignment 10 in xv6. The assignment covers four fundamental memory management topics:

1. Physical memory allocation tracking via memstats()
2. Virtual-to-physical address translation via vtop()
3. Page permission inspection and write protection via pageinfo() and setreadonly()
4. Comparative simulation of Page Replacement Algorithms (FIFO, LRU, Optimal)

All system calls and user programs were compiled and verified inside QEMU (qemu-system-i386) using an x86_64-elf-gcc cross-compiler on macOS.

2. Question 1 — Tracking Physical Memory Allocation

Kernel Implementation (kalloc.c & sysproc.c)

- **Page Counters:** Modified kalloc() and kfree() in kalloc.c to maintain counters for total_allocated and total_freed pages under the allocator lock (kmem.lock).
- **Initialization:** Seeded total_allocated during boot to count physical pages assigned to the kernel prior to user-space execution.
- **System Call (memstats):** Created sys_memstats() in sysproc.c to compute:
    - $CurrentlyAllocatedPages = TotalAllocated - TotalFreed$
- $$\text{Available Memory Pages} = \text{Count of free 4KB frames on } \texttt{kmem.freelist}$$

Program Verification (memtest.c)

The user program memtest allocates memory lazily via sbrk(), touches each page to trigger allocation, prints statistics, frees the pages with sbrk(-3 * 4096), and prints updated statistics.

Plaintext

```
None
After allocating and touching 3 pages:
Memory pages: allocated=989 freed=710 current=279 available=56786
```

```
After releasing the pages:
Memory pages: allocated=989 freed=713 current=276 available=56789
```

- **Analysis:** Releasing 3 pages increases freed from 710 to 713, reduces current from 279 to 276, and increases available from 56,786 to 56,789. The total managed physical memory remains constant at $276 + 56,789 = 57,065$ pages ($\approx 222.9MB$), reflecting physical RAM between end and PHYSTOP (0xE0000000).

3. Question 2 — Virtual-to-Physical Address Translation

Kernel Implementation (vm.c & sysproc.c)

- **System Call (vtop):** Implemented vtop(int virtual_address) in sysproc.c.
- **Page Table Traversal:** Uses walkpgdir() in vm.c to locate the Page Table Entry (PTE) for the given virtual address (va).
- **Offset & Frame Extraction:** Validates that va < KERNBASE (0x80000000) and PTE_P (Present) is set. Extracts the physical frame base address and appends the 12-bit page offset:
  $$\text{Physical Address} = \text{PTE_ADDR}(*pte) \mid (va \ \& \ 0\text{xFFF})$$
- **Failure Returns:** Returns -1 if the page is unmapped, invalid, or out of process bounds.

Program Verification (vttest.c)

Plaintext

```
None
va 0x3000 -> pa 0xDFBF000
va 0x307B -> pa 0xDFBF07B (same frame, nonzero offset)
unmapped lazy page 0x4000 -> -1
out-of-range address -> -1
```

- **Analysis:** Addresses 0x3000 and 0x307B point to the same physical frame (0xDFBF000), preserving the offset 0x07B. Unallocated lazy pages and invalid bounds correctly return -1.

4. Question 3 — Page Table Management and Protection

Kernel Implementation (vm.c, sysproc.c, trap.c)

- **pageinfo(va):** Locates the PTE for va, returning its frame number ($PA/4096$) and permission flags (PTE_P, PTE_W, PTE_U).

- **setreadonly(va):** Clears the PTE_W (Writable) bit in the PTE and reloads CR3 (lcr3(v2p(p->pgdir))) to flush stale TLB entries.
- **Protection Fault Interception (trap.c):** Intercepts exception vector 14 (T_PGFLT). If a write instruction targets a read-only page (err_code & FEC_WR), the kernel displays pid <id> <name>: protection page fault at addr <va>--kill proc and terminates the offending process.

Program Verification (pagetest.c)

Plaintext

```
None
Before protection:
va 0x3000: frame 57205 (pa 0xdf75000), P=1 W=1 U=1

After setreadonly:
va 0x3000: frame 57205 (pa 0xdf75000), P=1 W=0 U=1

Child attempting a write to the read-only page...
pid 6 pagetest: protection page fault at addr 0x3000--kill proc
Parent survived the child's protection fault.
```

- **Analysis:** setreadonly() changes permissions from W=1 to W=0. The child process attempt to write to 0x3000 triggers a hardware protection fault, killing the child while allowing the parent to reap it cleanly.

5. Question 4 — Page Replacement Algorithms

User Implementation (pagereplace.c)

- **FIFO (First-In First-Out):** Evicts the page that has been in memory the longest.
- **LRU (Least Recently Used):** Evicts the page whose last reference timestamp is the oldest.
- **Optimal (OPT):** Evicts the page whose next reference occurs furthest in the future (or is never referenced again).

Reference Sequence Performance (20 references)

Sequence: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

Plaintext

```
None
Frames Algorithm Faults Hits Hit ratio Fault ratio
```

```
3       FIFO      15      5     25.0%      75.0%
3       LRU       12      8     40.0%      60.0%
3       Optimal   9       11    55.0%      45.0%
4       FIFO      10      10    50.0%      50.0%
4       LRU       8       12    60.0%      40.0%
4       Optimal   8       12    60.0%      40.0%
5       FIFO      9       11    55.0%      45.0%
5       LRU       7       13    65.0%      35.0%
5       Optimal   7       13    65.0%      35.0%
```

Analytical Comparison

1. **Optimal Supremacy:** Optimal yields the minimum possible faults (9 faults with 3 frames) because it possesses future visibility of reference patterns.
2. **LRU Locality Exploitation:** LRU achieves 12 faults with 3 frames by retaining recently accessed pages (e.g., 0, 2, 3), outperforming FIFO.
3. **FIFO Limitations:** FIFO incurs the highest fault count (15 faults with 3 frames) because it evicts pages based purely on arrival age without regard for access frequency.